

TECHPULSE CAPSTONE PROJECT

Automotive Secure Code Debugging and Review Assistant

Authenticated repository review with evidence-grounded C/C++ analysis



Student	Karthikeya Bellapukonda
Register number	CB.AI.U4AID23109
University	Amrita Vishwa Vidyapeetham
Track / case study	Digital Track / Case Study 4
Artifact	Technical Report and Viva Study Guide — version 1.3
Submission date	06 October 2026

Executive summary

This capstone implements a local-first decision-support system for reviewing complete automotive C/C++ repositories. An engineer uploads an explicitly authorized ZIP. The service validates the archive without extracting or executing it, scans every supported source/header file within bounded limits, builds a repository tree and internal include-dependency map, and reports selected pointer, boundary, resource and control-flow risks with exact file, function, line and source evidence.

A private Retrieval-Augmented Generation (RAG) layer can retrieve relevant passages from an approved local PDF index and ask a locally hosted Ollama model to explain the highest-risk module. Unsupported citations fail closed. If the local model or index is unavailable, the application returns a clearly labeled deterministic fallback. SQLite stores input hashes, structured output and human-review events, but not source-code bodies. The system never claims MISRA compliance or autonomously approves code.

Claim	Submitted evidence
Real repository input	ZIP endpoint, all-file scan, module table, tree, include graph, skipped-file reasons
Runnable application	Streamlit UI, FastAPI backend, local runtime and Docker configuration
Traceable output	File/function/line evidence, severity, CWE, confidence, recommendation and status
Verified behavior	15 automated tests; five targeted defect cases; secured repository and SARIF scenarios
Responsible AI	Local model/index, citation post-check, fallback, no source persistence and human disposition

Contents

1. Case-study problem and requirements
2. What the finished system does
3. Architecture
4. Safe repository analysis
5. Deterministic static analysis
6. RAG explained completely
7. API, interface and persistence
8. Security, privacy and responsible AI
9. Evaluation
10. How to run and demonstrate
11. Limitations and roadmap
12. Viva revision guide

References

1. Case-study problem and requirements

Automotive embedded software contains safety- and cybersecurity-relevant logic, often written in C or C++. Reviews must consider pointer validity, memory bounds, resource ownership, control flow, compiler diagnostics and organizational coding guidance. Manual review is slow and reviewer-dependent. A public AI service can introduce intellectual-property leakage and unsupported answers. The design goal is therefore faster triage with local processing, evidence, bounded AI assistance and final human authority.

Case-study expectation	How this project meets it
Repository/code understanding	Scans all accepted C/C++ modules; summarizes functions, calls, branches, loops and lines
Cross-module context	Resolves quoted local includes and creates source-to-header dependency edges
Diagnostics and logs	Accepts warnings and structurally parses SARIF rule, severity, message and location evidence
Defect candidates	Detects selected pointer, boundary, resource and control-flow patterns
Standards explanation	Retrieves local approved passages and validates cited rule text
Structured results	Returns JSON findings with location, evidence, severity, confidence and remediation
Privacy	Local analysis/model path; no source body persisted in workflow database
Human validation	Accepted/Rejected/Needs changes disposition with reviewer, notes and audit time

The application is review assistance, not a compiler, certified static analyzer or compliance authority. Its output must be confirmed through compilation, tests, approved tools and qualified review.

2. What the finished system does

- Accepts a complete C/C++ repository as a ZIP plus optional warnings and build/runtime evidence.
- Validates the archive as untrusted input and never extracts or executes repository content.
- Scans every supported C/C++ file within explicit resource limits and records skipped-file reasons.
- Builds a tree, per-module inventory, internal quoted-include graph and unresolved-include list.
- Aggregates deterministic findings across files and preserves their original paths.
- Optionally explains the highest-risk module using approved local evidence and a local LLM.
- Exports the report as JSON and records an auditable human disposition.
- Retains a single-file mode for focused experiments and backward compatibility.

Repository limit	Value	Reason
Compressed archive	8 MB	Bounds upload and memory exposure
Supported source files	300	Prevents unbounded work
One source file	256 KB	Bounds decoding and analysis

Repository limit	Value	Reason
Total source text	2 MB	Bounds aggregate prompt/analysis data
Compression ratio	200:1	Rejects suspicious ZIP-bomb behavior

3. Architecture

Layer	Technology	Responsibility
Presentation	Streamlit	Upload, display maps/findings, export JSON, collect disposition
Service	FastAPI + Pydantic	Contracts, validation, API-key option, request audit and endpoint orchestration
Repository	repository_analyzer.py	Authorized-path ZIP validation, module inventory, tree, dependency graph, complexity and aggregation
Access control	API middleware + permission policy	Per-run authentication, repository ID verification and path-prefix authorization
CI integration	tools/ci_review.py + GitHub Actions	JSON/SARIF export and configurable severity gate
Evidence	static_analyzer.py	Repeatable source inventory and selected defect patterns
Retrieval	PyMuPDF + SentenceTransformers + ChromaDB	Extract, embed, store and retrieve approved PDF passages locally
Generation	LangChain + local Ollama	Explain retrieved evidence and return a structured review
Workflow	SQLite	Hashes, reports, status, notes, events and local metrics

Data moves from the UI to validated API models. Repository analysis runs before any optional AI call. The deterministic result therefore exists even if retrieval or generation fails. For RAG, the service chooses the module with the most findings, supplies bounded related context, and merges the grounded explanation with repository-wide findings. Only hashes and generated records cross the persistence boundary.

3.1 Main files

File	Purpose
src/ui/app.py	Three-tab interface for repository review, single-file review, history and metrics
src/api/main.py	Review endpoints, validation, error handling, storage and audit messages

File	Purpose
src/analysis/repository_analyzer.py	Archive boundary and repository-scale aggregation
src/analysis/static_analyzer.py	Per-file functions, control signals, calls and deterministic findings
src/rag/orchestrator.py	Retrieval, local model prompt, JSON validation, citation checking and fallback
src/ingestion/ingest.py	Approved PDF extraction, chunking, embedding and Chroma persistence
src/storage/review_store.py	SQLite schema and review/disposition event operations
tests/	API, storage, analyzer and malicious-archive checks

4. Safe repository analysis

4.1 Why ZIP input

A ZIP is portable and avoids storing Git provider credentials in the pilot. The launcher creates a random API secret and authenticated local identity. The request supplies a repository ID, which is checked against a repository and path-prefix allowlist before analysis. The archive remains the explicit content boundary. Provider-native checkout remains an organization integration because it requires real credentials and repository administration.

4.2 Validation sequence

1. Verify the archive is within the compressed-size budget and is readable as ZIP.
2. Inspect every central-directory entry before reading source content.
3. Reject null bytes, absolute paths, drive-qualified paths, parent traversal and duplicate normalized paths.
4. Reject encrypted entries, symbolic links and suspicious compressed-to-expanded ratios.
5. Ignore documented build, vendor, external, generated and tooling directories.
6. Accept supported C/C++ suffixes only; reject binary/null-containing or non-UTF-8 source.
7. Enforce per-file, total-source and accepted-file-count limits.
8. Analyze accepted text in memory; do not extract, import, compile or execute it.

4.3 Repository model

Each module record contains path, extension/language indicator, line count, functions, branch and loop counts, called functions, quoted includes, resolved targets and finding count. A quoted include is normalized relative to its source path. If the normalized target matches another accepted file, the system creates a dependency edge; otherwise it records an unresolved include. Angle-bracket library headers are intentionally not treated as internal repository edges.

Findings produced by the per-file analyzer are augmented with the repository path. Their identifiers are re-hashed with the path, preventing identical snippets in different files from sharing one ID. Aggregate output includes severity counts, affected-file count, total functions/lines, languages, root directories, dependency counts and skipped-file explanations.

4.4 Static analysis performance and CI evidence

SARIF 2.1 evidence is parsed into tool name, rule identifier, severity, message, file and line before it enters the review prompt. Repository output also reports complexity estimates, branch and loop totals, and dynamic

allocation observations for performance investigation. The CI command emits JSON or SARIF and returns a failing exit code at a configurable severity threshold. A GitHub Actions example demonstrates pull-request execution and SARIF upload.

4.5 Historical approved findings

When a reviewer marks a review Accepted, the storage layer can retrieve a bounded summary of its findings. Later RAG reviews receive these records in a separate human-approved context block. The model still requires current approved standard evidence for rule citations, so historical findings provide organizational experience without becoming an automatic compliance authority.

5. Deterministic static analysis

Deterministic means the same input and version produce the same rule-based output. The analyzer uses conservative text and control-pattern recognition. It is fast and explainable but cannot replace a compiler front end, abstract interpretation or interprocedural data-flow analysis.

Category	Example signal	Why it matters	Typical remediation
Pointer safety	Pointer dereference without an evident guard	Invalid access may crash or corrupt state	Prove non-null before dereference and handle failure
Boundary safety	strcpy-style unbounded copy	Destination overflow can corrupt memory	Use capacity-aware copy and prove termination
Resource management	Allocation without visible release	Leaks exhaust constrained embedded resources	Define ownership and release on every exit path
Control flow	switch without default	Unexpected state may have undefined behavior	Add an explicit safe default path

5.1 Finding contract

Every finding has a stable ID, title, category, severity, file, function, line, source evidence, recommendation, optional CWE, confidence, status and analysis method. This structure makes UI rendering, JSON export, evaluation and human review consistent. A candidate is not automatically a confirmed defect.

6. RAG explained completely

6.1 Definition

Retrieval-Augmented Generation (RAG) combines information retrieval with a generative language model. Instead of trusting the model's training memory, the application first searches an approved knowledge base for relevant text. It then supplies those passages to the model as evidence for the current question. This is augmentation at inference time; the model is not retrained.

6.2 Components and terms

Term	Meaning in this project
Chunk	A bounded passage extracted from an approved PDF, stored with source/page metadata
Embedding	A numeric vector representing the semantic meaning of text

Term	Meaning in this project
Vector similarity	A distance comparison used to find passages whose meaning resembles the query
Vector store	ChromaDB database containing vectors, original chunks and metadata
Retriever	Returns the five most relevant local chunks for a review query
Prompt	Instructions plus retrieved evidence and delimited untrusted code/log context
LLM	Ollama-hosted llama3.1 generating a contextual structured explanation
Grounding	Constraining claims to supplied evidence instead of model memory alone

6.3 Ingestion and retrieval

9. PyMuPDF extracts text and page metadata from approved PDFs.
10. The ingester divides text into retrievable chunks while preserving source information.
11. all-MiniLM-L6-v2 converts each chunk into an embedding locally; `local_files_only=True` forbids runtime download.
12. ChromaDB persists the collection locally with anonymized telemetry disabled.
13. At review time, the query is embedded using the same model and the five closest chunks are retrieved.
14. Retrieved passages, source and diagnostics are labeled as untrusted data inside bounded prompt sections.
15. The local model returns JSON that is parsed against a Pydantic schema.
16. A post-check verifies that any cited rule appears in retrieved text; otherwise citation/confidence fail closed.

6.4 Why this exact design

Automotive standards are controlled, versioned and frequently licensed. Fine-tuning would be expensive, hard to update and would not automatically provide inspectable sources. A public hosted model conflicts with the confidentiality objective. Local RAG allows the organization to approve the document set, refresh knowledge by re-indexing, preserve source/page metadata, and use a general local model only for explanation. The small all-MiniLM model is suitable for efficient semantic search; ChromaDB is lightweight and local; Ollama makes local model hosting accessible; temperature zero improves reproducibility.

6.5 Failure behavior

RAG is optional. If the embedding model, vector index or Ollama is unavailable, the service returns deterministic-fallback mode and never invents a successful retrieval. Repository-wide deterministic scanning still runs. The optional model focuses on the highest-risk module plus at most twenty related files so prompt size and latency remain bounded. This is a deliberate scope boundary, not a claim that the LLM analyzed only one-file repository input.

7. API, interface and persistence

Method	Endpoint	Purpose
GET	/health	Liveness
POST	/api/v1/repository-review	Safe complete-repository review
POST	/api/v1/review	Focused single-file review
GET	/api/v1/reviews	List history
GET	/api/v1/reviews/{id}	Read one report and audit events
PATCH	/api/v1/reviews/{id}/disposition	Record reviewer decision and notes
GET	/api/v1/metrics	Workflow counts

The Streamlit interface mirrors the contract: repository upload and evidence inputs; safe-boundary explanation; summary metrics; repository tree; module and dependency tables; skipped files; structured findings; downloadable JSON; and a human disposition form. It also applies an explicit high-contrast light theme to text, inputs, tabs, metrics, alerts, tables and buttons, preventing unreadable foreground/background combinations when browser or Streamlit defaults differ. History shows the persisted status without storing the source body.

7.1 Persistence and audit

SQLite stores a review ID, request/input hashes, mode, structured report, timestamps, status, reviewer name, notes and audit events. Hashes support traceability: the same bytes produce the same SHA-256 value. A hash does not reveal the source body and is not an encryption substitute. Application logs record metadata such as request ID, route, duration and finding counts, not submitted source text.

8. Security, privacy and responsible AI

Risk	Implemented control	Residual production action
ZIP traversal/bomb/link	No extraction; path/link/encryption/duplicate/size/count/ratio validation	Malware gateway and hardened isolated worker
Source leakage	Local path, local embeddings/model, source body excluded from SQLite/logs	Network isolation, disk encryption and retention policy
Prompt injection	Repository/log content labeled and delimited as untrusted data	Adversarial testing and content policy
False rule citation	Citation must appear in retrieved context; otherwise no-rule/Low result	Verify against licensed authoritative standard
Over-reliance	Advisory wording, visible limitations and separate human disposition	Training, RBAC, monitoring and override review
Unsafe automation	No automatic merge, release, compliance or safety approval	Formal safety case before any gated integration

8.1 Responsible-AI principles

- Transparency: report analysis mode, confidence, evidence and limitations.
- Human agency: a qualified reviewer owns the final disposition.
- Privacy: minimize retention and keep inference/retrieval local.
- Robustness: preserve deterministic behavior when optional AI fails.
- Accountability: record versioned artifacts, hashes, reviewer notes and events.
- Honesty: do not call synthetic guidance official MISRA or small tests production proof.

9. Evaluation

9.1 Automated verification

Fifteen Pytest checks pass. They cover health, input validation, response contracts, persistence, disposition, metrics, finding location, repository dependencies, archive traversal and symbolic-link rejection, structured SARIF parsing, path authorization, authenticated repository access and performance observations. The one warning is an upstream Starlette/AnyIO deprecation notice and not a failed application behavior.

9.2 Targeted defect benchmark

Case	Target	Result
NULL-001	Pointer safety	Pass
BOUND-001	Boundary safety	Pass
RESOURCE-001	Resource management	Pass
CONTROL-001	Control flow	Pass
SAFE-001	No target finding	Pass

For this narrow five-case set only: TP=4, FP=0, FN=0, precision=1.00, recall=1.00 and F1=1.00. This is functional evidence, not a general accuracy claim.

9.3 Repository acceptance scenario

Measure	Expected	Observed
Supported source files	3	3
Internal dependency edges	2	2
Findings	4	4
Affected files	1	1
Unresolved local includes	0	0
Vendor source	Excluded	Excluded with reason

The live UI was also exercised against the sample ZIP. It displayed 3 files, 44 lines, 2 functions, 2 dependency edges and 4 findings. The screenshot is from the running application, not a mockup.

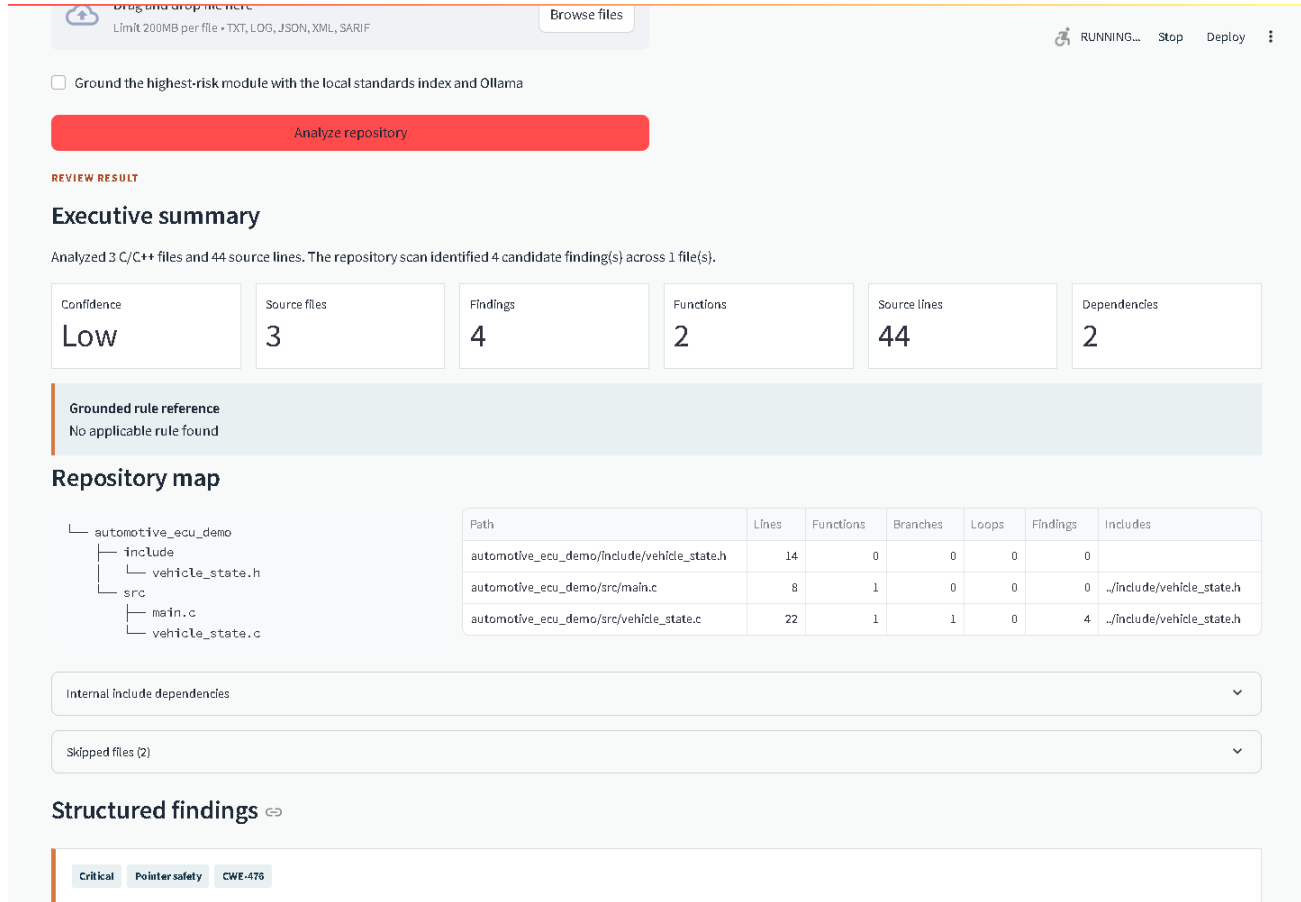


Figure 1. Live repository review showing summary metrics, repository tree, module table and structured findings.

10. How to run and demonstrate

- Create a Python 3.11+ virtual environment and install requirements.txt.
- Optional RAG: install Ollama, pull llama3.1, generate/replace approved guidance and run the ingestion script.
- Run python run.py; open <http://localhost:8501> and keep /docs open for the API contract.
- Upload the Automotive ECU Repository sample in Repository ZIP review and disable grounding if the local model/index is unavailable.
- Explain archive safety, 3 scanned modules, 2 edges, ignored vendor file and 4 path-specific findings.
- Download JSON, record Needs changes with a reviewer note, then show history and metrics.
- Run pytest and both evaluation scripts to show reproducibility.

Sample ZIP	Expected teaching point
AutomotiveECURepository	Four findings, two edges, ignored vendor content
CleanSpeedMonitor	Repository mapping with no target deterministic findings
MixedSensorNetwork	Multiple modules, shared header, boundary finding, ignored generated content

11. Limitations and roadmap

11.1 Honest limitations

- Regex/pattern analysis can miss defects and report false positives; it does not parse full C/C++ semantics.
- Include mapping does not model macro expansion, conditional compilation or complete compiler search paths.
- The submitted PDF guidance is synthetic demonstration content, not licensed MISRA material.
- The RAG call is bounded to one high-risk module plus limited context, while deterministic analysis covers all accepted files.
- The benchmark is deliberately small and cannot estimate industrial performance or safety suitability.
- The pilot authenticates local API requests and verifies repository/path permissions, but provider-native Git checkout requires organization-managed credentials.

11.2 Roadmap

- Use Clang AST and control/data-flow analysis alongside the implemented SARIF integration.
- Connect the implemented permission model to provider-native Git identity, commit pinning and organization project isolation.
- Add OIDC/RBAC, encrypted storage, secrets management, retention controls and signed artifacts.
- Evaluate on a large labeled automotive corpus with per-category quality and confidence calibration.
- Promote the implemented CI severity gate only after the organization defines benchmark thresholds, human approval and rollback controls.

12. Viva revision guide

Likely question	Full-mark answer
What is your project?	A local repository-scale C/C++ review assistant: safe ZIP mapping, deterministic evidence, optional grounded local AI and auditable human disposition.
Why repository ZIP?	It demonstrates complete repository analysis while keeping authorization explicit and avoiding Git credentials in the pilot.
Does it execute uploaded code?	No. It reads validated UTF-8 source in memory and never extracts, imports, compiles or executes archive content.
What is RAG?	Retrieve relevant passages from an approved local index, supply them as evidence, then generate an explanation grounded in those passages.
Why not just an LLM?	LLMs may hallucinate and are non-deterministic. Rules provide repeatable exact evidence and fallback; the LLM only explains context.
What are embeddings?	Numeric semantic representations used to match a query with relevant guidance even without exact keyword equality.
Why ChromaDB?	A lightweight local vector store that keeps embeddings, original chunks and metadata without a cloud service.
How do you stop fake rules?	Post-validate that the cited text occurs in retrieved evidence; otherwise return no applicable rule and Low confidence.

Likely question	Full-mark answer
How is privacy protected?	Local model/retrieval, source excluded from DB/logs, only hashes and reports persisted, explicit upload boundary.
What did tests prove?	The submitted contracts, target patterns, repository mapping, authentication, path permissions, SARIF parsing and archive defenses work in tested cases, not production accuracy.
Is this MISRA compliant?	No. It is advisory. Formal compliance requires licensed standards, approved tooling and qualified reviewers.
Best innovation?	The combination of safe all-file repository mapping, deterministic graceful degradation, fail-closed citations and human-in-command auditability.
Next improvement?	Clang AST and data flow, then connect the existing SARIF, permission and CI controls to enterprise identity, encrypted storage and a large benchmark.

References

[1] Tata Technologies, Automotive Engineering AI Project Case Studies, Version 1.0, September 2026, Case Study 4 and common guidance.

[2] TechPulse Program, Digital Track capstone submission and evaluation instructions, 2026, supplied course documents.

[3] Submitted project source, test suite, evaluation ground truth/results, README and USER_GUIDE.md.